Jarvis Protocol Architecture Brief

Jarvis Protocol Architecture Brief

Human-agent collaboration and learning-loop protocol

Jarvis defines protocol records for governed, reviewable, attributable, evidence-backed, portable human-agent work that improves across WorkSessions.

Jarvis is a protocol. Hosts own implementation.

Scope: v0.1 first 30 days, with future interoperability context.

Executive Summary

Jarvis is the protocol for governed human-agent collaboration and shared learning. It defines how a HumanWorker and an AgentWorker coordinate inside a WorkSession under shared goals and human-defined Policy, create Requests when the AgentWorker is blocked, record Reviews and Takeovers from human judgment, preserve attributable Contributions, export portable EvidenceManifest records, and carry governed LearningRecords, MemoryProposals, and SkillProposals into future WorkSessions.

The architecture is protocol architecture. It describes stable records, operations, boundaries, and conformance expectations. It does not describe a deployment architecture.

> v0.1 locks the protocol contract, examples, conformance entry, and compatibility boundary. Host implementation work stays outside the v0.1 core until the OpenAPI contract and conformance gate are stable.

Architecture Principles

Principle	Meaning
Protocol-only	Jarvis defines portable collaboration records. Hosts own execution.
Human-agent team	The valuable unit is the HumanWorker and AgentWorker working together.
Policy-governed autonomy	AgentWorker action becomes protocol state only after PolicyDecision.
Human judgment central	Requests, Reviews, and Takeovers preserve human authority.
Evidence during work	EvidenceManifest references work evidence captured during the WorkSession.
Attributable contribution	Human, agent, pair, and service contributions remain distinguishable.
Governed learning	LearningRecords, MemoryProposals, and SkillProposals do not silently mutate durable state.
Runtime agnostic	Hosts keep their execution systems. Jarvis records collaboration.

Research Grounding

Co-Gym reinforces the Jarvis thesis: human-agent collaboration needs dual control, bidirectional communication, and non-turn-taking coordination. It also shows why v0.1 must be strict about Requests and situational awareness without turning every coordination idea into a core object.

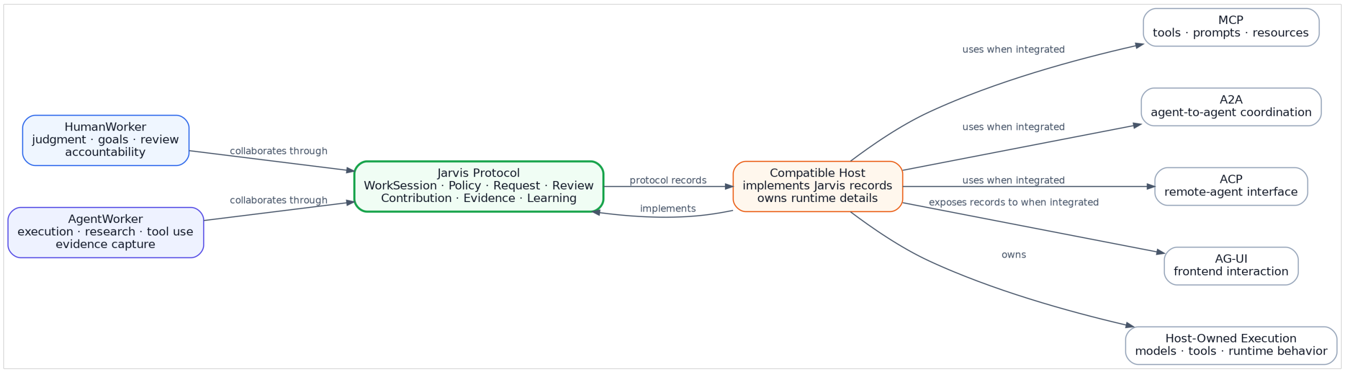
Jarvis applies that lesson directly:

```
        v0.1 defines the spine.
        Future versions expand the coordination layer.
```

The v0.1 spine prevents wrong implementation of policy, Request, Review, Takeover, Contribution, Evidence, and Learning. Notification protocols, collaboration metrics, observation visibility, and richer coordination acts are future extensions.

C1: Protocol Ecosystem Context

The context view shows Jarvis as the protocol record between HumanWorkers, AgentWorkers, hosts, and adjacent protocols.

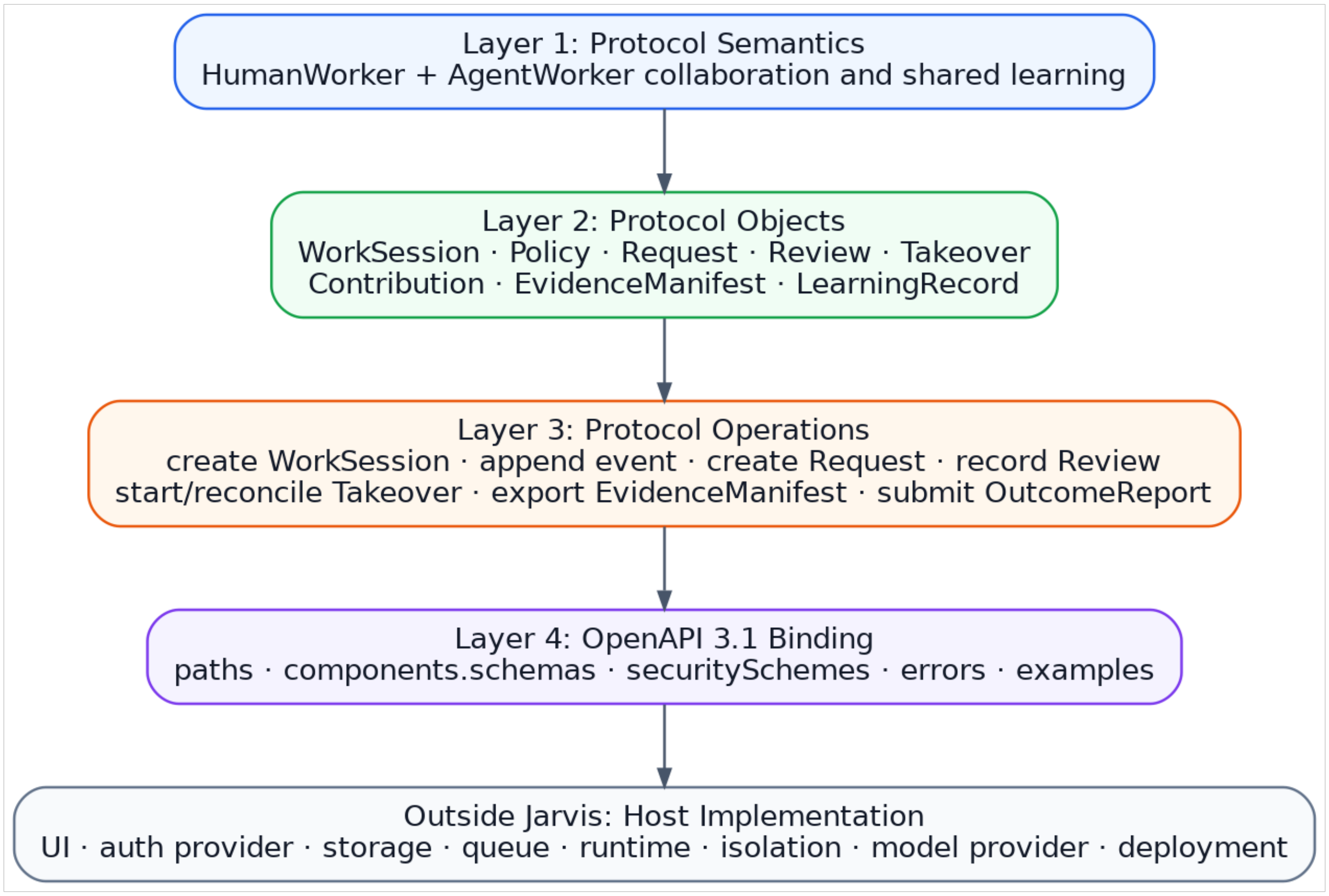


What This Means

- HumanWorkers and AgentWorkers collaborate through WorkSessions.
- Hosts implement Jarvis records.
- Host-owned execution remains outside Jarvis.
- MCP, A2A, ACP, and AG-UI remain external protocols.
- Jarvis does not own frontend rendering, agent runtime, remote-agent execution, tool protocol semantics, storage, auth provider, isolation, or deployment.

C2: Protocol Layer View

Jarvis separates protocol semantics, protocol objects, protocol operations, and OpenAPI communication binding. Host implementation stays outside Jarvis.



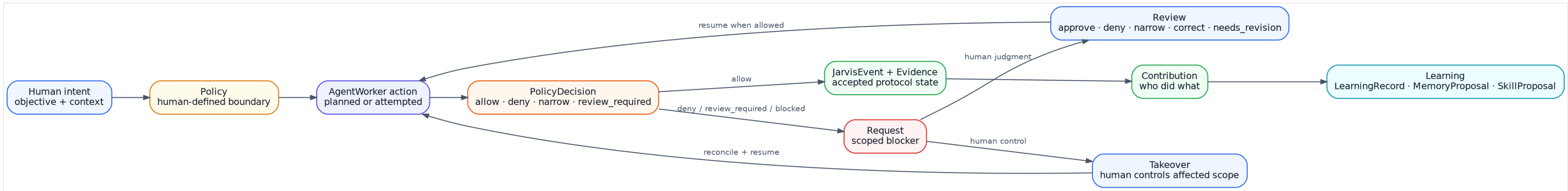
Layer Responsibilities

Layer	Responsibility
Protocol semantics	HumanWorker and AgentWorker collaboration and shared learning.
Protocol objects	WorkSession, Policy, Request, Review, Takeover, Contribution, EvidenceManifest, LearningRecord, MemoryProposal, SkillProposal, and related records.
Protocol operations	Create WorkSession, append event, create Request, record Review, start Takeover, record Contribution, export EvidenceManifest, submit OutcomeReport.
OpenAPI binding	HTTP paths, schemas, security scheme, header parameters, errors, examples, and conformance entry.
Host implementation	

Layer	Responsibility
	UI, auth provider, storage, queue, runtime, isolation mechanism, model provider, deployment, and host workflow.

WorkSession Control Flow

The control flow shows how agent autonomy, policy, human judgment, evidence, and learning connect inside one WorkSession.

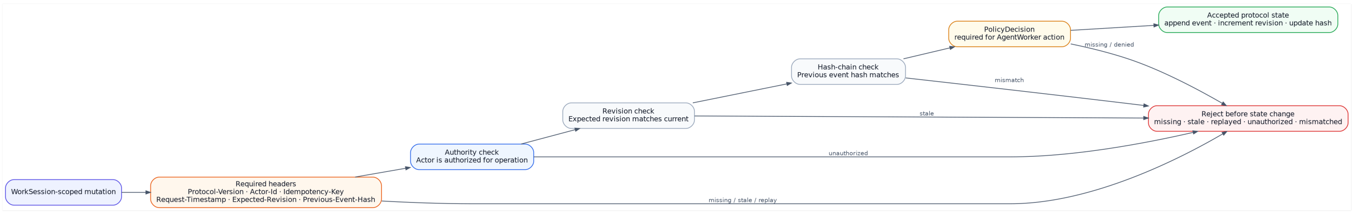


Control Flow Invariants

- WorkSession is the source of truth.
- AgentWorker action that affects a WorkSession records PolicyDecision before acceptance as protocol state.
- Action inside Policy records JarvisEvent and evidence.
- Action outside Policy creates a scoped Request.
- Request is not chat, notification, or authority.
- Human resolution requires Review or Takeover.
- Review decisions are approve, deny, narrow, correct, takeover, and needs_revision.
- Takeover creates a lock epoch and rejects stale autonomous continuation.

Zero-Trust Mutation Gate

Jarvis starts from zero trust. Compatible implementations reject unsafe mutations before protocol state changes.



Required WorkSession-Scoped Mutation Headers

```
Jarvis-Protocol-Version
Jarvis-Actor-Id
Jarvis-Idempotency-Key
Jarvis-Request-Timestamp
Jarvis-Expected-WorkSession-Revision
Jarvis-Previous-Event-Hash
```

Non-WorkSession protocol mutations use the non-WorkSession mutation header set:

```
Jarvis-Protocol-Version
Jarvis-Actor-Id
Jarvis-Idempotency-Key
Jarvis-Request-Timestamp
```

Worker registration, Actor registration, and OutcomeReport submission do not require fake WorkSession revision or previous event hash values.

Evidence And Learning Flow

Evidence and learning are not afterthoughts. They are part of the collaboration record.

Evidence And Learning Invariants

- Contribution records who did what.
- EvidenceManifest references events, policy decisions, requests, reviews, takeovers, contributions, artifacts, evidence items, and limitations.
- Evidence is captured during work.
- LearningRecord captures what the human, agent, or pair learned.
- MemoryProposal and SkillProposal require governed review state.
- Unreviewed model or tool output cannot silently become durable memory or active skill.

Adjacent Protocol Boundary

Jarvis integrates with adjacent protocols without replacing them.

Adjacent system	What it owns	What Jarvis records
MCP	Tools, prompts, resources, and context servers.	Tool/resource use as PolicyDecisions, JarvisEvents, Contributions, and Evidence.
A2A	Agent-to-agent communication and delegation.	Delegation evidence, participating AgentWorker refs, Contributions, and outcomes.
AGNTCY ACP	Remote agent interface and OpenAPI access.	Collaboration record around remote agent participation.
AG-UI	Agent-to-frontend event interaction and UI state.	WorkSession records that a host exposes to a frontend when frontend integration exists.
Agent execution systems	Model calls, tool calls, tracing, handoffs, runtime behavior.	Policy, review, contribution, evidence, and learning around execution.
Developer execution systems	Terminal, editor, repository, and command execution workflows.	WorkSession evidence, human reviews, takeovers, and learning.
Host-owned interfaces	The user-facing or service-facing interface around work.	Protocol semantics for governed HumanWorker and AgentWorker collaboration.

> Jarvis adoption fails if it requires a developer to abandon existing execution systems, UI, model providers, isolation mechanisms, storage, or deployment. Jarvis adoption succeeds when different hosts produce the same protocol semantics.

v0.1 First 30 Days

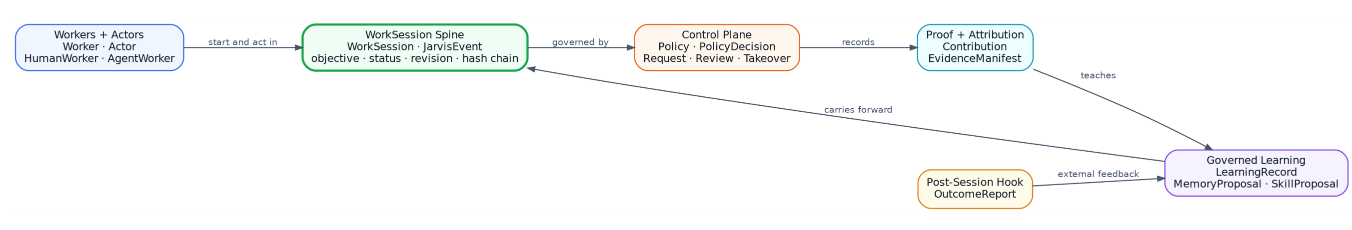
The first 30 days produce a usable protocol contract.

Area	v0.1 Work
`components.schemas`	Encode Worker, Actor, HumanWorker, AgentWorker, WorkSession, JarvisEvent, Policy, PolicyDecision, Request, Review, Takeover, Contribution, EvidenceManifest, LearningRecord, MemoryProposal, SkillProposal, and OutcomeReport.
`paths`	Encode create WorkSession, append event, record PolicyDecision, create Request, record Review, start/reconcile Takeover, record Contribution, create learning records, export EvidenceManifest, and submit OutcomeReport.
`securitySchemes` + `parameters`	Encode `HostAuth` as the security scheme and encode ActorHeader, ProtocolVersionHeader, IdempotencyHeader, RequestTimestampHeader, RevisionHeader, and PreviousHashHeader as header parameters.
Errors	Encode protocol error ids and required error envelope.
Examples	Provide first examples for WorkSession, Request, Review, and EvidenceManifest export.
Conformance	Define golden-path and failure-mode checklists that reject fake implementations.

v0.1 encodes the locked protocol. It does not redesign the thesis, object model, lifecycle, control plane, evidence model, learning model, security entry, or positioning boundary.

v0.1 Spine

v0.1 includes the minimum protocol objects required to prove the collaboration loop:




```
 Worker
Actor
HumanWorker
AgentWorker
WorkSession
JarvisEvent
Policy
PolicyDecision
```

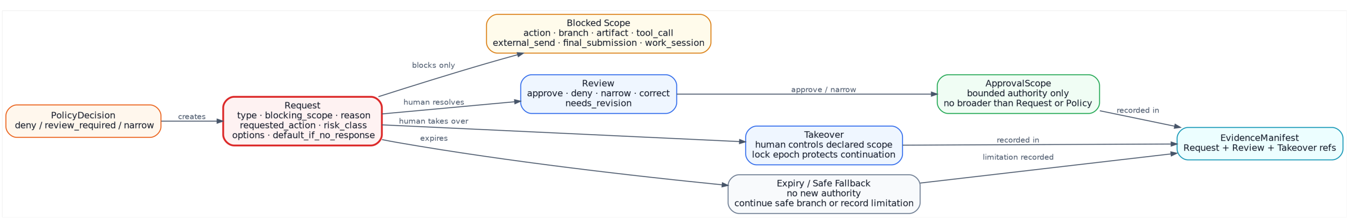
```
Request
Review
Takeover
Contribution
EvidenceManifest
LearningRecord
MemoryProposal
SkillProposal
OutcomeReport
```

v0.1 includes the core protocol operations:

```
create WorkSession
append event
record PolicyDecision
create Request
record Review
start/reconcile Takeover
record Contribution
create LearningRecord
export EvidenceManifest
submit OutcomeReport
```

Request Correctness

Request is the v0.1 control-plane object that prevents fake collaboration.



The Request schema must preserve:

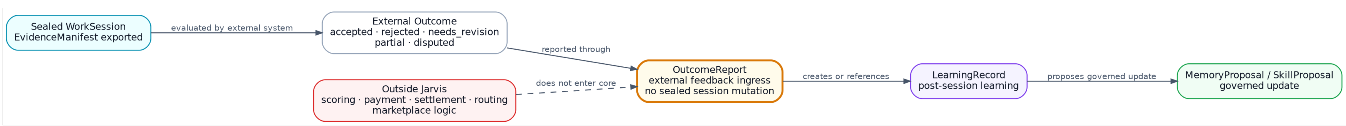
```
type
blocking_scope
reason
requested_action
risk_class
policy_decision_id
options
recommended_option
default_if_no_response
expires_at
resolution_ref
```

Request conformance must prove:

```
Policy-denied action creates Request.
AgentWorker cannot execute blocked action before resolution.
Request cannot resolve without Review or Takeover.
Narrowed approval prevents execution outside approved scope.
Takeover rejects stale AgentWorker continuation.
Expired Request follows safe fallback.
Request appears in EvidenceManifest.
```

OutcomeReport Hook

OutcomeReport is the v0.1 external feedback ingress. It carries post-session outcome information into governed learning without turning Jarvis into external evaluation, payment, settlement, routing, or marketplace logic.



Minimal example:

```
External task reviewer rejects submission
  -> OutcomeReport submitted
  -> LearningRecord created
  -> MemoryProposal or SkillProposal proposed
```

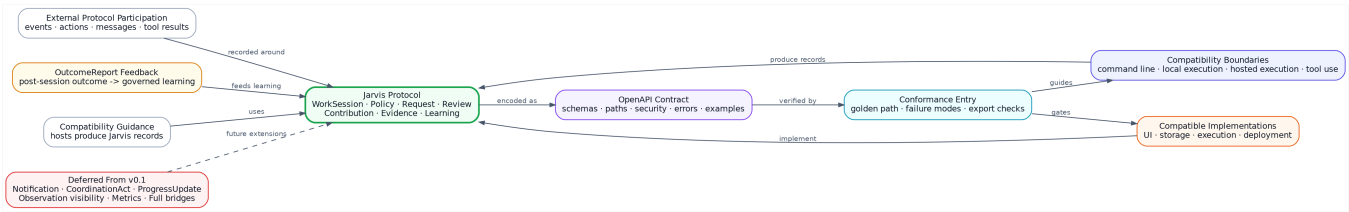
Protocol Error Entry

The OpenAPI error model must include the protocol errors that protect the v0.1 spine. These include:

```
missing_policy
missing_policy_decision
request_unresolved
missing_review_resolution
missing_takeover_resolution
invalid_approval_scope
stale_takeover_epoch
invalid_previous_event_hash
duplicate_idempotency_key_mismatch
unauthorized_actor
missing_evidence_event_refs
silent_memory_mutation
silent_skill_activation
outcome_report_without_learning_record
```

Future Interoperability Direction

Future work expands around the protocol without moving execution ownership into Jarvis.



Future area	Direction	Boundary
Compatibility boundaries	Map command-line, local execution, hosted execution, and tool-use flows into Jarvis records.	Hosts preserve their own execution systems.
Host conformance	Provide golden-path and failure-mode fixtures that prove protocol behavior.	Conformance checks behavior, not infrastructure.
Compatibility guidance	Map host work into Jarvis WorkSessions and EvidenceManifest exports.	The host remains the host; Jarvis remains the protocol.
External protocol bridges	Record MCP, A2A, ACP, and AG-UI participation as Jarvis evidence and contribution records.	Jarvis does not redefine those protocols.
Evaluation feedback	Use OutcomeReport to carry external outcomes into governed LearningRecords.	Jarvis does not own task routing, scoring, payment, settlement, or marketplace logic.
Public protocol package	Publish OpenAPI contract, examples, conformance checklist, and protocol architecture brief.	Adoption does not require a runtime owned by Jarvis.

Deferred From v0.1

The following ideas are important but stay outside the v0.1 core:

```
Notification object
CoordinationAct object
ProgressUpdate
Observation visibility model
CollaborationMetrics
InitiativeBalance
TeachingSignal
HumanGrowthRecord
SimulatedHumanWorker
full host adapters
full external outcome feedback loop
compatibility examples
```

```
        MCP/A2A/ACP/AG-UI bridges
        multi-agent reviewer protocol
```

These remain future extensions, examples, adapters, or conformance fixtures. They do not enter the v0.1 object spine.

Scope Boundary

Jarvis does not own:

- UI
- authentication provider
- authorization backend
- storage
- queues
- model provider
- tool execution
- isolation mechanism
- local execution
- cloud provider
- deployment
- billing
- host-specific workflow
- host execution
- host UI
- host storage
- external evaluation logic

Jarvis owns the protocol records that compatible hosts implement.

References

- Collaborative Gym: https://arxiv.org/abs/2412.15701
- MCP specification: https://modelcontextprotocol.io/specification/
- A2A specification: https://a2a-protocol.org/latest/specification/
- AGNTCY ACP specification: https://spec.acp.agntcy.org/
- AG-UI documentation: https://docs.ag-ui.com/
- OpenAPI 3.1.1: https://spec.openapis.org/oas/v3.1.1.html

Closing

Jarvis v0.1 succeeds when a compatible implementation runs a HumanWorker and AgentWorker through a WorkSession, proves policy-governed autonomy, captures human judgment, records contribution, exports evidence, and carries governed learning into the next WorkSession without adopting a runtime owned by Jarvis.